```
$cars = ['Golf', 'Polo'];
addCar($cars);
print_r($cars);
```

```
Output:
Array
(
    [0] => Golf
    [1] => Polo
    [2] => Scirocco
)
```

Notice that the `$inputArr` parameter now has the reference operator in front. This tells PHP that any changes should affect the original data that is passed in by the calling code. Now when we call `addCar()`, PHP does not create a copy of `$cars`, and the incoming data can be modified.

There also exists a mechanism for returning by reference, though it is a bit more cumbersome. Let's imagine that we want to keep count of something across the whole of our project, perhaps we want to tally every call to a database, something like that. We could, of course, use a global variable with all that entails (spoilers: global vars should be kept to an absolute minimum). There is a way, however, to achieve this using a function.

We can have our function return a reference to a `static` variable. A static variable will keep its value even when it goes out of scope at the end of function execution. We will examine the use of static variables in functions in much more detail later in this chapter when we explore the concept of *scope*. To get this working we first declare the function with an `&` character prepending the function name. Then we declare the locally scoped static variable, and finally we return it.

```
function &getCounter()
{
    static $i = 0;
    return $i;
}
```

Notice the `static $i = 0;` statement. Although it reads as if `$i` will be set to zero on every call, PHP will only execute this statement once (the first time the function is called). It's a little syntactic sugar to avoid any unwieldy `if` statements. Delicious!